

{EPITECH}

Poké-Bot



Poké-Bot

Introduction



Language: JavaScript



✓ La documentation JavaScript nécessaire pour réaliser le bot est disponible sur le site **discord.js**

✓ En cas de question, pensez à **demander de l'aide** à votre voisin de droite. Puis de gauche. Ou inversement. Puis demandez enfin à un encadrant si vous êtes toujours bloqué(e).

Cet été, une chaleur inhabituelle s'abat sur le monde des Pokémons. À Illumis, les températures grimpent, et pour aider les habitants à mieux vivre cette canicule, la directrice du camping a une idée originale : créer un **bot Discord** pour animer les journées et proposer des activités ludiques.

Le problème, c'est qu'elle n'y connaît rien en programmation... Heureusement, elle a déjà discuté du projet avec le maire, qui a tout de suite pensé à vous : le développeur de confiance de la ville. Il souhaite que vous preniez en charge cette mission.

Seule demande un peu particulière de la directrice : le bot doit être à l'image de son Pokémon préféré... **Carapuce** !

I. Installation des prérequis

Pour réaliser cette mission, vous aurez besoin d'installer **Node.js**, un environnement d'exécution JavaScript côté serveur.

Ouvrez le terminal, et faites la commande **"winget install -e --id OpenJS.NodeJS"**.

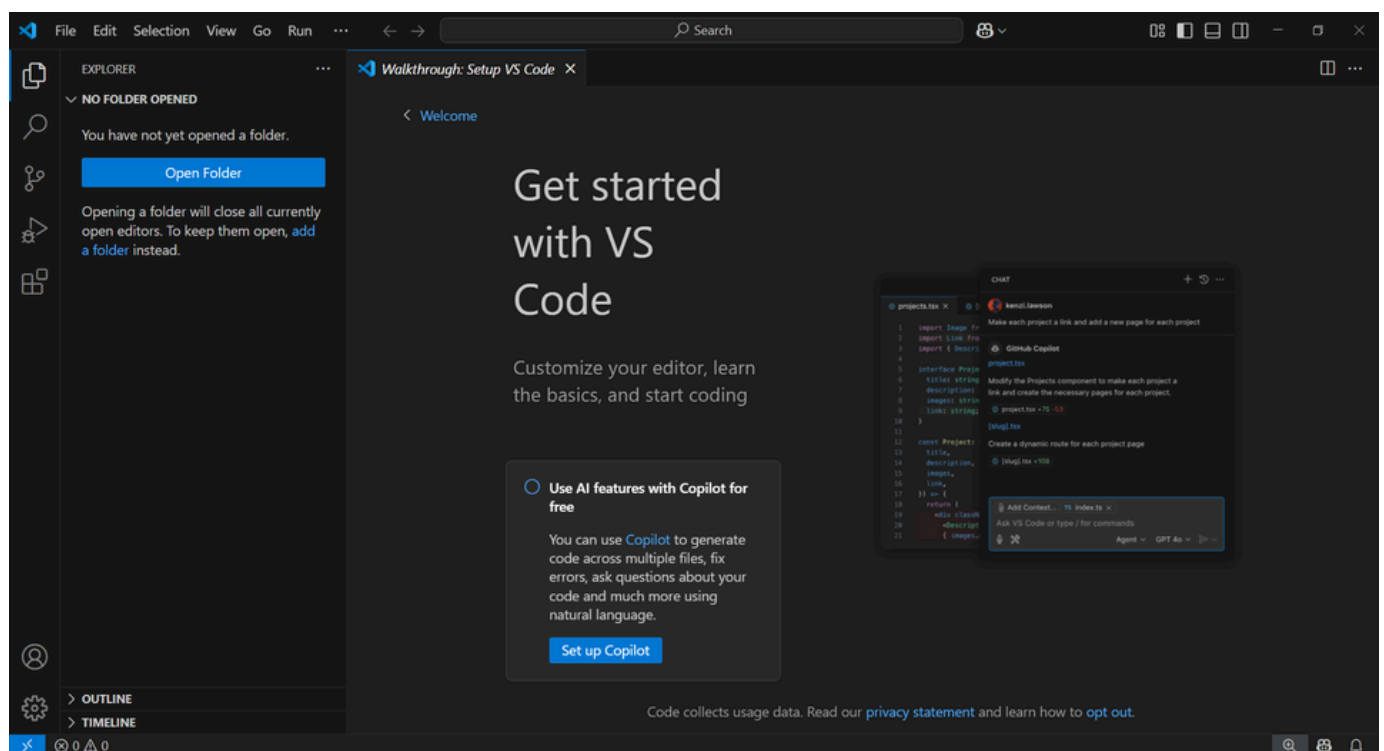
Une fois la tâche finie, faites les commandes **"node -v"** et **"npm -v"** pour vérifier que tout a été installé correctement. Si les deux commandes renvoient une version, vous avez réussi ! Vous pouvez fermer le terminal.

Il vous faudra également un éditeur de texte pour développer confortablement votre bot.

Vous pouvez choisir d'utiliser le **bloc-notes** de Windows, même si personne ne vous le conseillera.

Vous pouvez également choisir d'utiliser **Notepad++**, sa version améliorée qui ajoute de la couleur et qui est peut-être déjà installée sur votre ordinateur. *Après tout, pourquoi pas.*

Sinon, vous pouvez choisir le confort d'un éditeur de code, comme par exemple **Visual Studio Code** - (**"winget install -e --id Microsoft.VisualStudioCode"**)

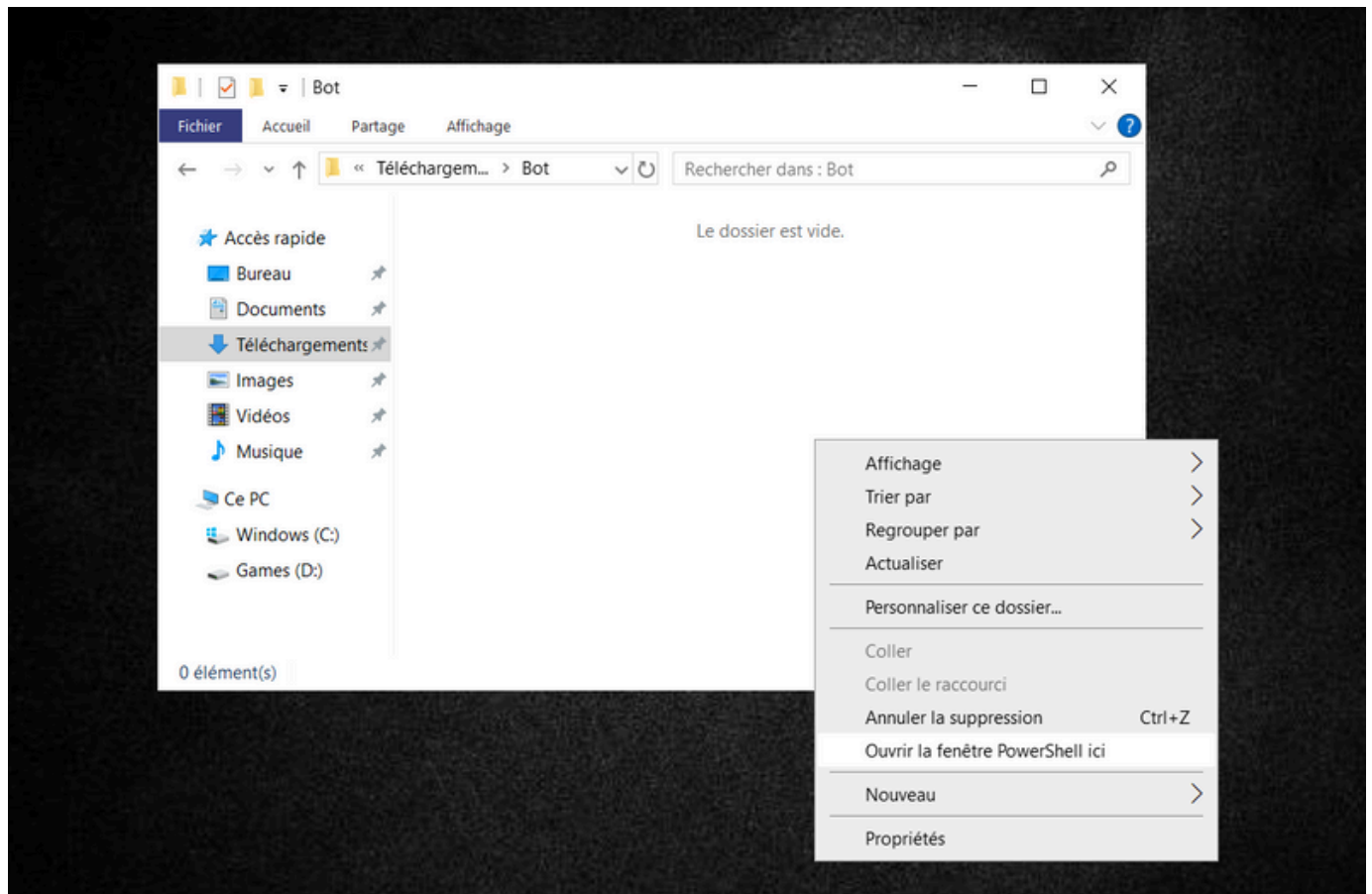


II. Mettre en place le répertoire de travail

Maintenant, on va créer la base de notre bot.

Créez un nouveau dossier sur votre ordinateur et rentrez à l'intérieur de celui-ci avec votre explorateur de fichiers.

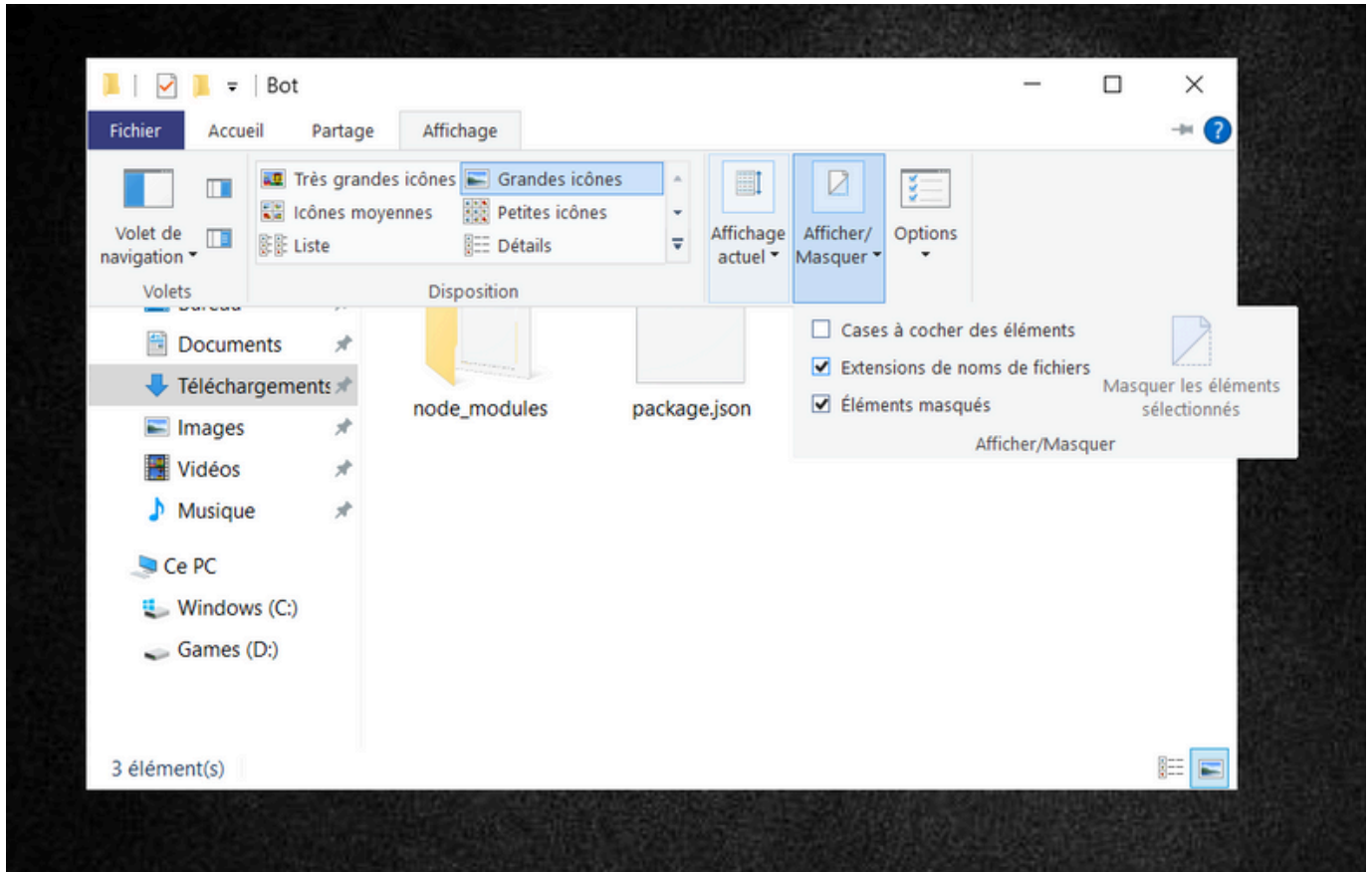
Appuyez sur la touche **MAJ** et faites, en même temps, un **clic droit** avec votre souris puis sélectionnez *“Ouvrir la fenêtre PowerShell ici”* dans le menu déroulant.



Faites la commande **“npm install discord.js”** puis refermez le terminal.

```
PS C:\Users\Yui-MHCP001\Downloads\Bot> npm install discord.js
added 25 packages in 6s
7 packages are looking for funding
run 'npm fund' for details
```

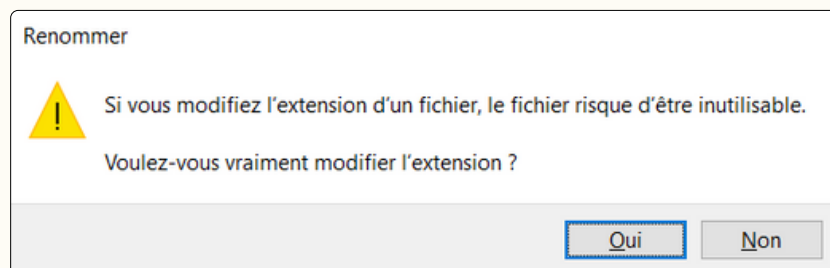
Activez la visibilité des extensions de nom de fichier dans l'explorateur Windows



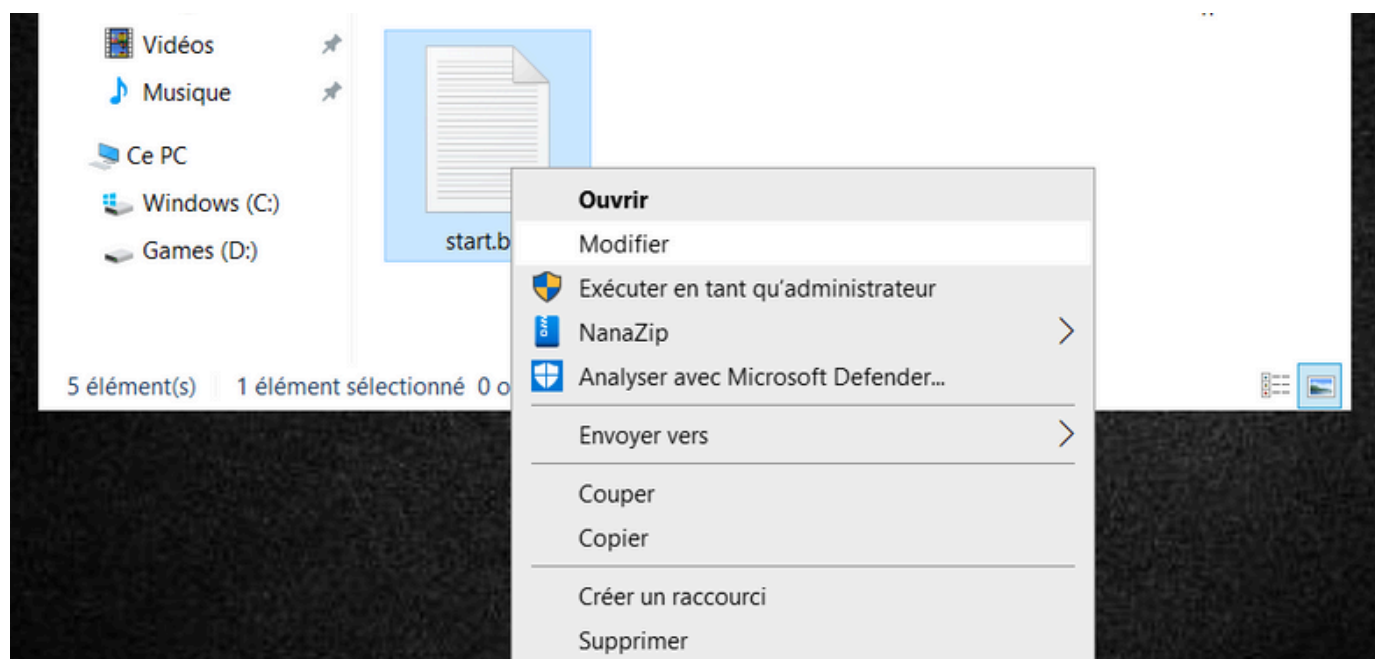
On pourrait démarrer notre bot en utilisant le terminal mais pour rendre ça plus accessible, on va créer un exécutable. Dans votre dossier, créez un fichier **start.bat**.



Il est probable que Windows vous affiche cette boîte de dialogue. Il ne s'agit que d'une mise en garde qui ne concerne pas vraiment les formats de textes. Sélectionnez oui et le fichier sera créé.



Faites un **clique droit** avec votre souris sur le fichier **start.bat** puis sélectionnez “*Modifier*” dans le menu déroulant.



Écrivez “**node index.js**” à l’intérieur puis sauvegardez et fermez le fichier.

Maintenant, refaites la même opération : créez un fichier **update.bat** et écrivez “**node deploy-command.js**” à l’intérieur. Il servira à déclarer à l’API discord les modifications de vos **/commande**.



Si vous êtes du côté IDE de la force, pensez à ouvrir le dossier de votre bot dans l’éditeur, ça vous fera économiser du temps.

La mise en place est maintenant terminée. Il est temps de passer aux choses sérieuses !

III. Initialisation



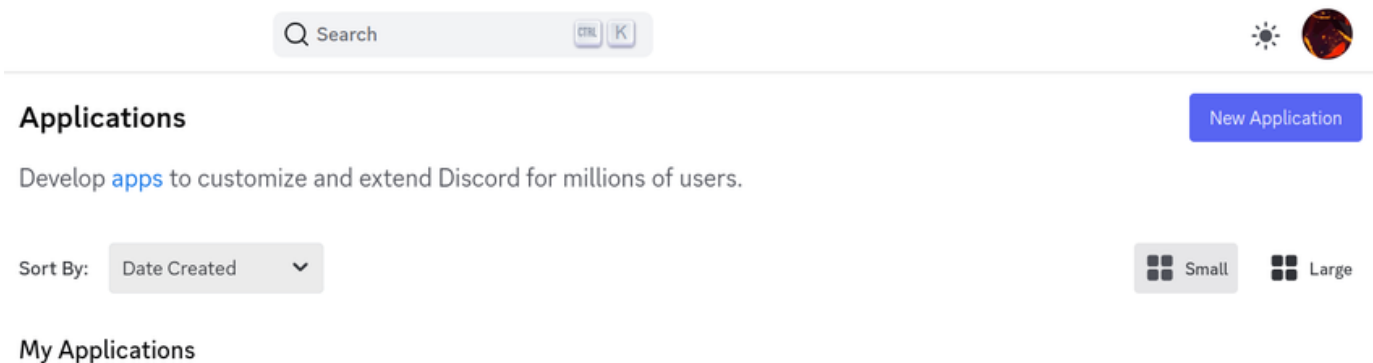
Pour cette activité, vous aurez besoin d'avoir un **compte Discord** et d'y être **connecté**. Si vous n'en avez pas encore, commencez par en créer un.

Nous avons désormais un bot discord, ou du moins ses fichiers.

Pour pouvoir l'utiliser, il va d'abord falloir le déclarer sur le **Discord Developer Portal** en créant une application.



Le **Discord Developer Portal** contient également toute la documentation de l'API discord, que nous utiliserons durant cette activité.



Créez une nouvelle application, puis donnez-lui un nom.



Dans la page **Bot** des paramètres, vous pouvez modifier à tout moment le nom de votre bot et son image. Le changement peut prendre jusqu'à 2 minutes.

[← Back to Applications](#)

SELECTED APP

Carapuce ▼

SETTINGS

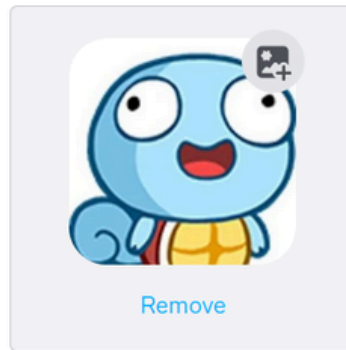
General Information

Installation NEW

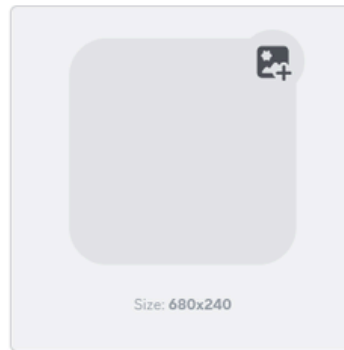
OAuth2

Bot

ICON



BANNER



USERNAME

Carapuce

Pour connecter notre bot à discord, on va avoir besoin de récupérer certaines informations (mettez-les de côté dans un **bloc-notes**, par exemple) :

- le **Token** : le mot de passe de connexion de votre bot à Discord (vous devez le **réinitialiser** pour pouvoir le copier).

TOKEN

For security purposes, tokens can only be viewed once, when created. If you forgot or lost access to your token, please regenerate a new one.

MTM3OTc0NDQxMzY2MzYyNTI2Ng.GJK0qy.jlw0Qd3NBKOxrvJ1BPuFKmMfVmkMGpolnWgNpQ

Copy

Reset Token



Posséder votre **Token**, ça signifie pouvoir se connecter à votre bot et donc effectuer des actions en votre nom à votre insu. Ne le donnez à **personne**.

Retournez sur la page des informations générales de l'application.

- l'**Application ID** : identifie votre bot auprès de l'API Discord.
- la **Public Key** : permet de vérifier la signature des interactions reçues, afin d'éviter toute falsification.

APPLICATION ID

1379744413663625266

Copy

PUBLIC KEY

9a45ea4b900daa5e6d6317bd601a974becb197986100e7c490131700b77015d3

Copy

Retournez sur la page du bot.

On va maintenant lui donner toutes les permissions nécessaires.

On coche **Presence Intent**, **Server Members Intent** et **Message Content Intent** puis on oublie pas de sauvegarder.

PRESENCE INTENT

Required for your bot to receive [Presence Update](#) events.

NOTE: Once your bot reaches 100 or more servers, this will require verification and approval. [Read more here](#)



SERVER MEMBERS INTENT

Required for your bot to receive events listed under [GUILD_MEMBERS](#).

NOTE: Once your bot reaches 100 or more servers, this will require verification and approval. [Read more here](#)



MESSAGE CONTENT INTENT

Required for your bot to receive [message content](#) in most messages.

NOTE: Once your bot reaches 100 or more servers, this will require verification and approval. [Read more here](#)



! Careful — you have unsaved changes!

Reset

Save Changes

Allez sur la page OAuth2.

On y retrouve un **générateur de lien d'invitation**, qui permet de créer une URL pour ajouter son bot Discord à son serveur, avec les permissions souhaitées.

Les applications Discord ne sont pas nécessairement des bots. Il faut donc préciser au préalable que c'est un bot, en cochant la **case** correspondante.

OAuth2 URL Generator

Generate an invite link for your application by picking the scopes and permissions it needs to function. Then, share the URL to others!

SCOPES

☐ identify	☐ email	☐ connections
☐ guilds	☐ guilds.join	☐ guilds.members.read
☐ guilds.channels.read	☐ gdm.join	☒ bot
☐ rpc	☐ rpc.notifications.read	☐ rpc.voice.read
☐ rpc.voice.write	☐ rpc.video.read	☐ rpc.video.write
☐ rpc.screenshare.read	☐ rpc.screenshare.write	☐ rpc.activities.write
☐ webhook.incoming	☐ messages.read	☐ applications.builds.upload
☐ applications.builds.read	☐ applications.commands	☐ applications.store.update
☐ applications.entitlements	☐ activities.read	☐ activities.write
☐ activities.invites.write	☐ relationships.read	☐ relationships.write
☐ voice	☐ dm_channels.read	☐ role_connections.write
☐ presences.read	☐ presences.write	☐ openid
☐ dm_channels.messages.read	☐ dm_channels.messages.write	☐ gateway.connect
☐ account.global_name.update	☐ payment_sources.country_code	☐ sdk.social_layer_presence
☐ sdk.social_layer	☐ lobbies.write	
☐ applications.commands.permissions.update		

Un second menu apparaît alors et on coche les bonnes cases pour donner uniquement les permissions nécessaires au bot : ça permet de limiter ses utilisations potentiellement détournées.

BOT PERMISSIONS

GENERAL PERMISSIONS

- ☐ Administrator
- ☐ View Audit Log
- ☐ Manage Server
- ☒ Manage Roles
- ☐ Manage Channels
- ☐ Kick Members
- ☐ Ban Members
- ☒ Create Instant Invite
- ☒ Change Nickname
- ☒ Manage Nicknames
- ☒ Manage Expressions
- ☒ Create Expressions
- ☒ Manage Webhooks
- ☒ View Channels
- ☒ Manage Events
- ☒ Create Events
- ☐ Moderate Members
- ☐ View Server Insights
- ☐ View Server Subscription Insights

TEXT PERMISSIONS

- ☒ Send Messages
- ☐ Create Public Threads
- ☐ Create Private Threads
- ☒ Send Messages in Threads
- ☐ Send TTS Messages
- ☒ Manage Messages
- ☐ Manage Threads
- ☒ Embed Links
- ☒ Attach Files
- ☒ Read Message History
- ☐ Mention Everyone
- ☒ Use External Emojis
- ☒ Use External Stickers
- ☒ Add Reactions
- ☒ Use Slash Commands
- ☒ Use Embedded Activities
- ☒ Use External Apps
- ☒ Create Polls

VOICE PERMISSIONS

- ☒ Connect
- ☒ Speak
- ☐ Video
- ☐ Mute Members
- ☐ Deafen Members
- ☐ Move Members
- ☒ Use Voice Activity
- ☐ Priority Speaker
- ☐ Request To Speak
- ☒ Use Embedded Activities
- ☐ Use Soundboard
- ☐ Use External Sounds

On obtient finalement un lien d'invitation pour pouvoir enfin rencontrer notre bot !

INTEGRATION TYPE

Guild Install

GENERATED URL

https://discord.com/oauth2/authorize?client_id=1379744413663625266&permissions=1716213066886209&integration_type=0&scope=bot

Copy

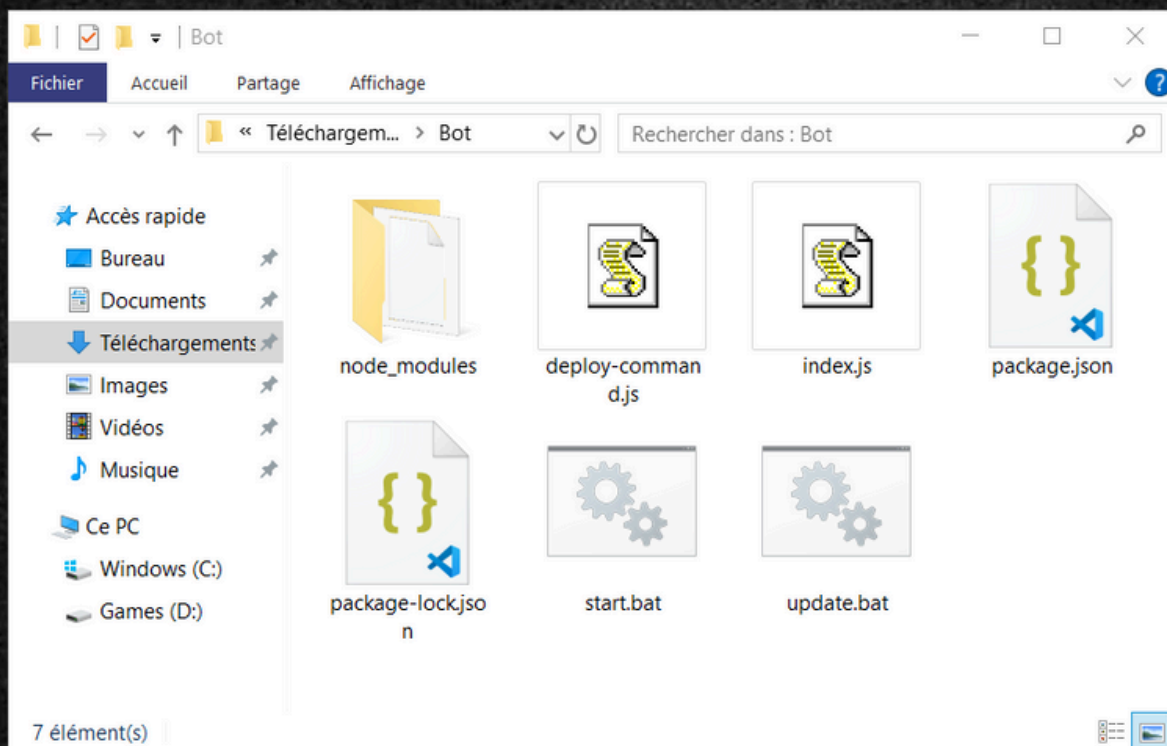
Il n'y a plus qu'à ouvrir le lien dans le navigateur et à ajouter le bot sur le serveur Discord de l'activité.

IV. Premier échange avec Carapuce

Carapuce a rejoint notre serveur mais il est marqué hors-ligne et ne répond pas à notre appel quand on essaie de lui parler.

Après avoir demandé conseil à ce bon vieux Frankenstein, il semblerait qu'il manque encore le plus important : le code.

Récupérez les fichiers **index.js** et **deploy-command.js** fournis avec le sujet et ajoutez les dans le dossier de votre bot. Si tout s'est bien passé jusque là, votre dossier devrait logiquement ressembler à ceci.



Ouvrez les fichiers que vous venez de télécharger et essayez de comprendre comment les utiliser à l'aide des commentaires.

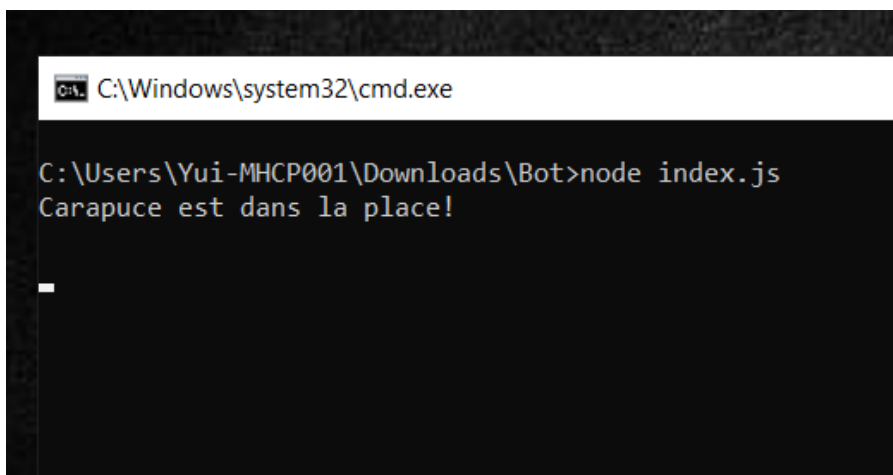
Si vous avez été suffisamment attentif, vous avez sûrement remarqué les champs à compléter **VOTRE_TOKEN** et **VOTRE_APPLICATION_ID** qu'il va falloir modifier avec les informations que vous avez récupéré précédemment.

```
await rest.put(  
  Routes.applicationCommands("VOTRE_APPLICATION_ID"),  
  { body: commands },  
);
```

```
await rest.put(  
  Routes.applicationCommands("1379744413663625266"),  
  { body: commands },  
);
```

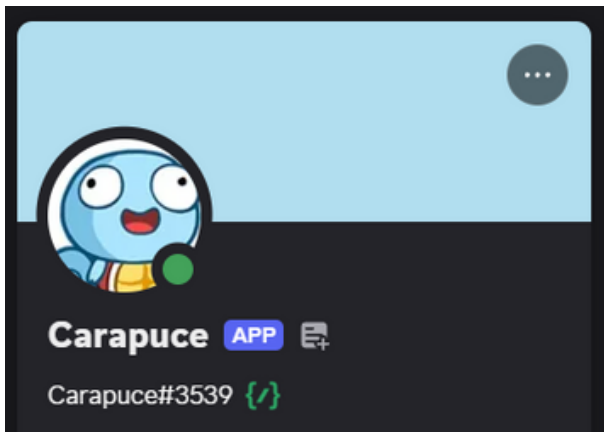
Votre bot est maintenant complet, il est temps de le démarrer.

Ouvrez l'explorateur de fichier et **double cliquez** sur les fichiers **update.bat** et **start.bat**. Si tout a bien fonctionné, vous devriez voir le message *"Carapuce est dans la place!"* dans un terminal qui vient de s'ouvrir.

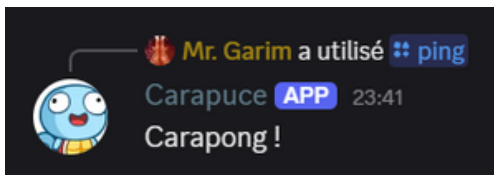


```
C:\Windows\system32\cmd.exe  
  
C:\Users\Yui-MHCP001\Downloads\Bot>node index.js  
Carapuce est dans la place!  
  
_
```

Vous devriez également voir Carapuce en état **connecté** sur Discord. Si c'est le cas, bien joué à vous !



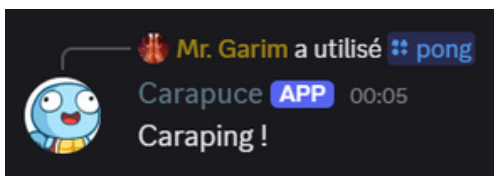
Essayez maintenant de faire **/ping** dans un salon où le bot est présent. Si vous pouvez le sélectionner et qu'il vous renvoie "Carapong !" tout fonctionne correctement.



Un bot Discord analyse chaque message et réagit lorsqu'il correspond à l'une de ses actions prédéfinies. Il peut interagir avec le serveur de différentes façons : **envoyer des messages, modifier des rôles**, ou encore **effectuer des actions de modération** sur les utilisateurs.

Dans le cas des **/commande**, il effectue des opérations directement via l'API discord et aucun message n'est réellement envoyé.

En analysant le fonctionnement du code fourni, essayez maintenant de créer une commande **/pong** qui renvoie "Caraping !".





Si vous modifiez une commande existante, vous devez redémarrer le bot en fermant le terminal, puis en relançant **start.bat**.

Si vous ajoutez une nouvelle **/commande**, il faut également exécuter **update.bat** pour que Discord la prenne en compte. Actualisez également la page de votre navigateur si vous n'utilisez pas l'application Discord.

Maintenant que vous avez compris comment fonctionnent les **/commande**, vous êtes libre de continuer de suivre ce sujet ou d'essayer de développer vos propres commandes par vous-même, en vous aidant de la **documentation**, de **YouTube** et des **IA** (ChatGPT, Gemini, etc...).

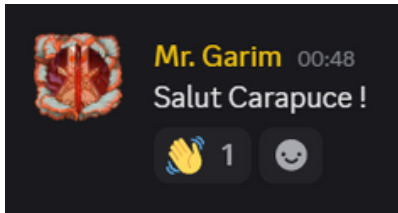
Tout est possible quand on a suffisamment d'imagination !

V. Étendre les connaissances de Carapuce

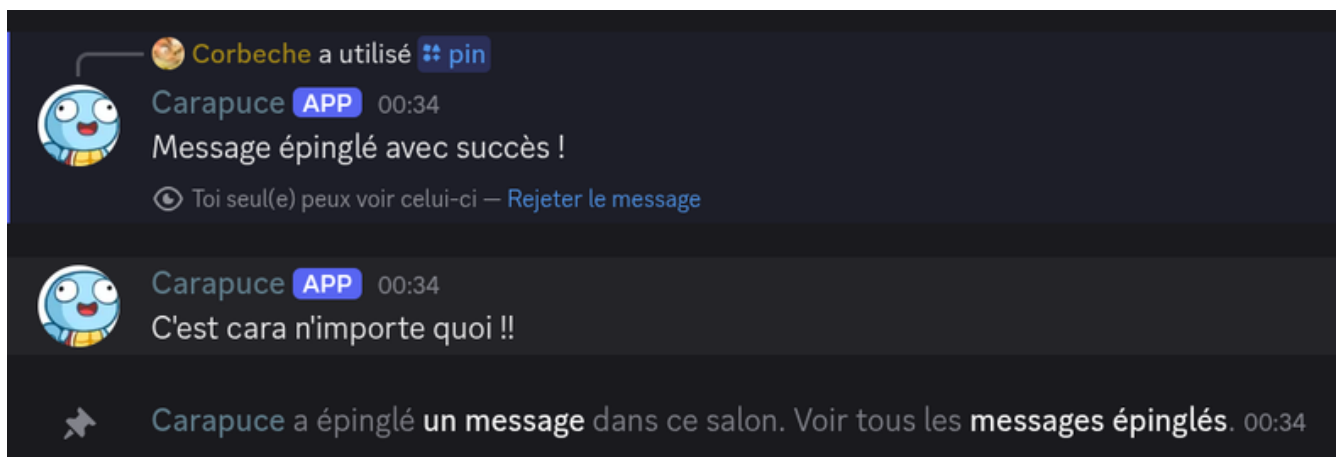
Quoi ? Vous êtes toujours là ? Vous en voulez encore ? Très bien. C'est parti !

Les messages sont l'un des éléments centraux de Discord, et on va en tirer parti.

Commençons par faire en sorte que Carapuce **réagisse** avec un emoji “👋” chaque fois qu'un message commence par "Salut".



Ensuite, passons à la création de messages épinglés. Pour cela, on va définir une commande **/pin [message]**, qui prend un seul argument : le texte du message. Le bot devra l'envoyer dans le salon, puis l'épingler automatiquement.



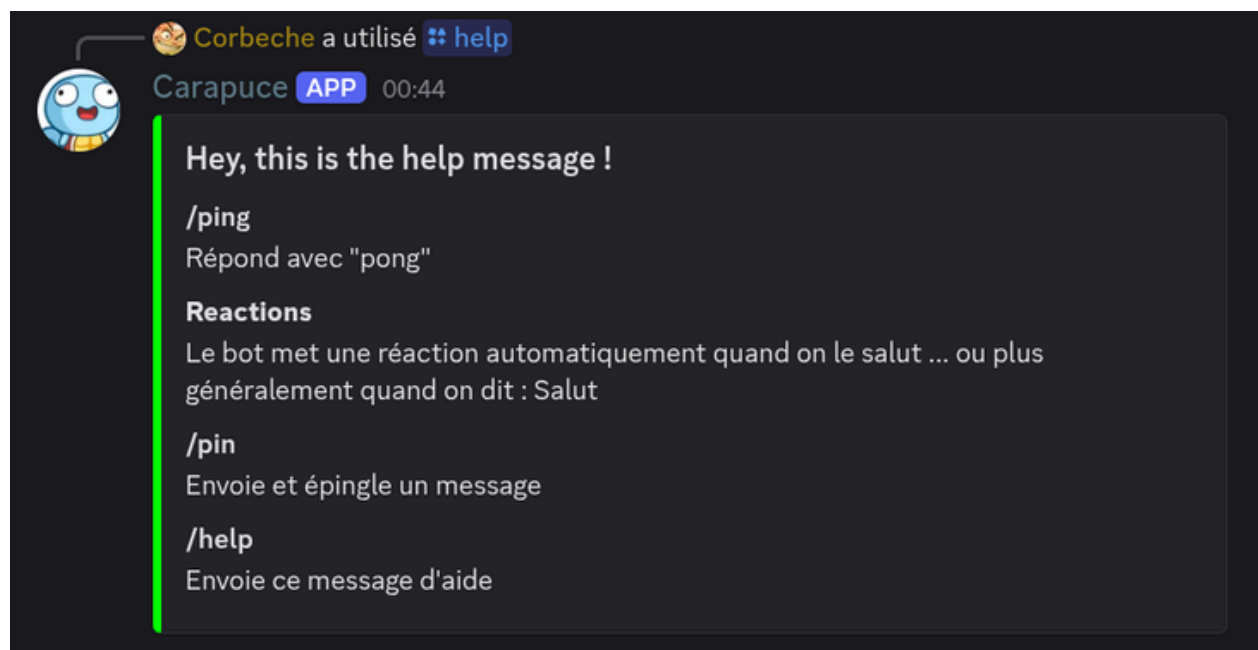
Dans notre exemple, le message épinglé par le bot n'est pas une réponse directe à la commande. En revanche, le bot répond bien avec un message visible uniquement par vous qui semble ... **éphémère** ?

Pour qu'une interaction entre le bot et l'utilisateur soit considérée comme valide, le bot doit toujours répondre à la commande, même brièvement. Sans réponse, Discord considère l'interaction comme échouée.

Ok, on peut maintenant épingler des messages, recevoir un pong, et Carapuce nous dit bonjour.

Mais à force d'ajouter des fonctionnalités, la directrice du camping Pokémon risque de s'y perdre...

Pas de panique, il est temps d'ajouter une commande indispensable à tout bot digne de ce nom : la fameuse **/help**. Elle utilisera un nouveau type de message : les **Embeds**. C'est bien plus esthétique, et pratique ! Mais si, vous verrez !



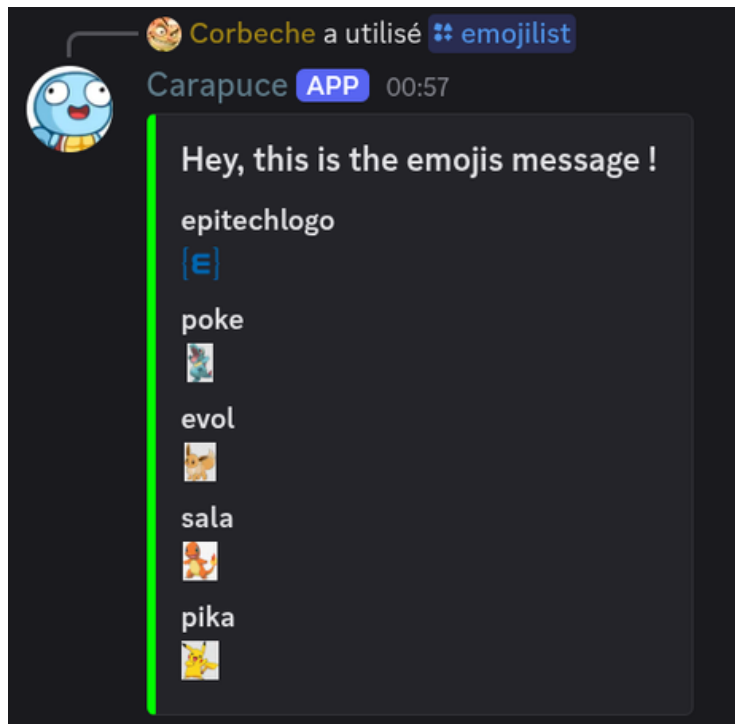
Magnifique ! Il ne faudra pas oublier d'alimenter cette commande au fur et à mesure de votre progression...

Et si on explorait maintenant une autre facette de Discord ? ... les emojis.

On en a déjà parlé plus tôt, mais saviez-vous que notre serveur possède ses propres emojis personnalisés ? Non ? Eh bien, il est temps de s'y intéresser !

Créez une commande **/emojilist** qui permet de voir tous les emojis du serveur et de connaître leur nom exact, pour pouvoir les utiliser facilement.

Pour que ce soit propre et agréable à lire, on va tout afficher dans un joli **embed**.
Et bien oui, on veut tout savoir... mais avec style !

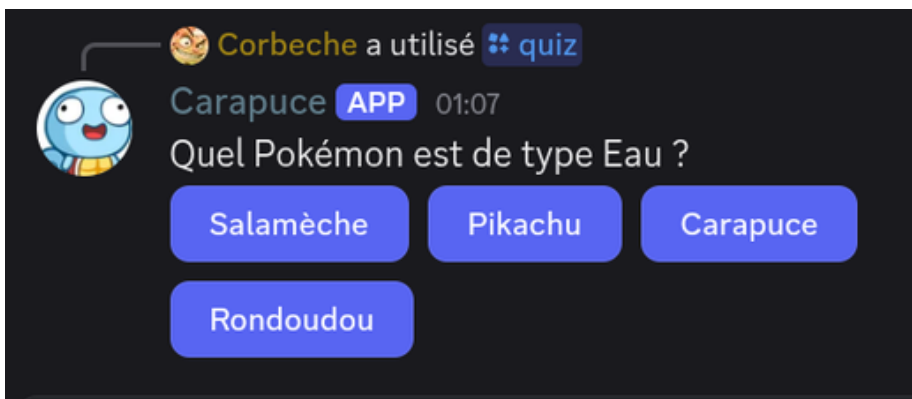


VI. Les /commande ... c'est tout ?

Vous et votre Carapuce maîtrisez désormais les **/commande** comme des champions. Vous êtes prêts à décrocher ce badge dans l'arène... Mais attention, il reste encore bien des défis avant de pouvoir affronter la Ligue Pokémon !

Des mécaniques comme les Buttons, Select Menus, Modals, Autocomplete Interactions, ou encore les Reaction Events ... sont autant de cordes que vous pouvez ajouter à votre arc pour parfaire votre bot, mais d'autres aspects sont à prendre en compte sur ces commandes ... Vous verrez ...

Commençons avec quelque chose de relativement "facile" : les boutons. Créez une **/commande** qui envoie un **ActionRow** qui contiendra une question et des propositions de réponses.



Un **ActionRow** correspond à un **bouton** dans la documentation de **discord.js**

Alors ? Facile ? Ok, ok, trop facile pour vous, mais est-ce que votre Carapuce peut identifier la bonne réponse ?



Corbeche a utilisé **quiz**

Carapuce **APP** 01:07
Quel Pokémon est de type Eau ?

Salamèche Pikachu Carapuce Rondoudou

Carapuce **APP** 01:10
Mauvaise réponse.

Toi seul(e) peux voir celui-ci — [Rejeter le message](#)

Carapuce **APP** 01:10
Bonne réponse.

Toi seul(e) peux voir celui-ci — [Rejeter le message](#)

Encore trop simple ? Alors, est-ce que votre bot peut choisir une **question aléatoire** parmi une liste stockée dans un **fichier externe** ? **Mélanger** les choix de réponses ?

```
1  [
2    {
3      "question": "Quel Pokémon est de type Eau ?",
4      "choices": [
5        "Carapuce",
6        "Salamèche",
7        "Pikachu",
8        "Rondoudou"
9      ],
10     "answer": "Carapuce"
11   },
```



 Corbeche a utilisé  quiz

Carapuce  01:18

Combien de Pokémon de la première génération existe-t-il ?

152

100

150

151



 APP Carapuce Combien de Pokémon de la première génération existe-t-il ? 

Carapuce  01:18

Mauvaise réponse.

 Toi seul(e) peux voir celui-ci — [Rejeter le message](#)



 Corbeche a utilisé  quiz

Carapuce  01:18

Qui est le professeur Pokémon dans la première génération ?

Prof. Orme

Prof. Seko

Prof. Sorbier

Prof. Chen



 APP Carapuce Qui est le professeur Pokémon dans la première génération ? 

Carapuce  01:18

Bonne réponse.

 Toi seul(e) peux voir celui-ci — [Rejeter le message](#)



 Corbeche a utilisé  quiz

Carapuce  01:19

Quel est le nom de la région de la première génération ?

Hoenn

Johto

Kanto

Sinnoh



 APP Carapuce Quel est le nom de la région de la première génération ? 

Carapuce  01:19

Bonne réponse.

 Toi seul(e) peux voir celui-ci — [Rejeter le message](#)

VII. Vers l'infini et au-delà !

Si vous êtes arrivé jusqu'ici, il faut l'admettre : vous êtes un véritable ingénieur Pokémon en herbe. Mais les professionnels retravailleraient quelques aspects de leur bots ...

- **Stockage des variables sensibles** : Actuellement, votre Token, votre Application ID et d'autres informations sensibles sont **écrits en clair** dans votre code. Est-ce réellement la meilleure option ?
- **Bases de données** : Vous avez sûrement remarqué que des bots populaires comme DraftBot, MEE6 ou UnbelievaBoat **s'adaptent** à chaque serveur. Pourquoi ? Parce qu'ils utilisent une base de données. Cela leur permet d'enregistrer des réglages, des scores, des préférences utilisateurs, etc... Et même en cas de crash, les données restent sauvegardées.
- **Gestion des rôles** : Des commandes sensibles comme **/ban** ou **/kick** ne doivent pas pouvoir être utilisées par n'importe qui. Il faut donc que votre bot vérifie les rôles de l'utilisateur avant d'exécuter ce genre d'action.

Avec un peu de lecture de documentation, quelques recherches, et votre logique infaillible d'ingénieur Pokémon, votre bot pourra rivaliser avec les plus grands. Et tout ça sans débourser un seul centime dans un abonnement premium. Qui dit mieux ?

Carapuce vous félicite. Le chemin est encore long, mais vous avez déjà franchi de nombreuses épreuves.

Alors... prêt pour la Ligue ?

{EPITECH}

